

Folding

CSC345: Programming Languages & Paradigms

Another higher-order function: `foldr`

`foldr`: “fold right”

- Going to implement the operation of folding an operator or function into a list of values
- *Motivation*: lots of list library functions can be defined using a similar recursion pattern:

```
sum [] = 0  
sum (x:xs) = x + sum xs
```

```
or [] = False  
or (x:xs) = x || or xs
```

```
product [] = 1  
product (x:xs) = x * product xs
```

```
and [] = True  
and (x:xs) = x && and xs
```

foldr: fold right

```
sum [] = 0  
sum (x:xs) = x + sum xs
```

```
or [] = False  
or (x:xs) = x || or xs
```

```
product [] = 1  
product (x:xs) = x * product xs
```

```
and [] = True  
and (x:xs) = x && and xs
```

... these functions all follow the following simple pattern of recursion on lists:

```
f [] = v  
f (x:xs) = x # f xs
```

foldr: fold right

$$\begin{aligned} f [] &= v \\ f (x:xs) &= x \# f xs \end{aligned}$$

- Maps empty list to value v
- Maps nonempty list to head of list, with some operator $\#$, and the result of recursively processing the tail

- **foldr** encapsulates this pattern of recursion
- Requires 3 arguments:
 1. The operator $\#$
 2. The value v
 3. The **list of values** (unless using partial application, covered in the next lecture)

Rewriting functions using `foldr`

- **Prompt:** rewriting the `sum`, `product`, `or`, and `and` definitions (more compactly) using `foldr`:

```
sum :: Num a => [a] -> a
sum xs = foldr (+) 0 xs
```

```
product :: Num a => [a] -> a
product xs = foldr (*) 1 xs
```

```
or :: [Bool] -> Bool
or xs = foldr (||) False xs
```

```
and :: [Bool] -> Bool
and xs = foldr (&&) True xs
```

Definition of foldr

```
sum :: Num a => [a] -> a  
sum xs = foldr (+) 0 xs
```

```
or :: [Bool] -> Bool  
or xs = foldr (||) False xs
```

Function Definition:

```
foldr f v [] = v  
foldr f v (x:xs) = f x (foldr f v xs)
```

Function Signature:

(first guess)

```
foldr :: (a -> a -> a) -> a -> [a] -> a
```

Definition of `foldr`

Function Definition:

```
foldr f v [] = v  
foldr f v (x:xs) = f x (foldr f v xs)
```

Function Signature:

(first guess)

```
foldr :: (a -> a -> a) -> a -> [a] -> a
```

Function Signature:

(actual)

```
foldr :: (a -> b -> b) -> b -> [a] -> b
```

Example Trace

`foldr (+) 0 [1,2,3]`

Example: defining `reverse` using `foldr`

```
> reverse [1..5]  
5,4,3,2,1
```

- Need a “helper function” named `snoc`
 - (`cons` backwards)
 - `snoc` will add a new element to the *end of the list* rather than the *start*

```
cons :: a -> [a] -> [a]  
cons x xs = x:xs
```

```
snoc :: a -> [a] -> [a]  
snoc x xs = xs ++ [x]
```

Now defining `reverse`: **SOLUTION A**

- Uses recursion

```
reverse :: [a] -> [a]  
reverse [] = []  
reverse (x:xs) = reverse xs ++ [x]
```


Example: defining `reverse` using `foldr`

Now defining `reverse`: **SOLUTION B**

- Uses `snoc` and recursion

```
> reverse [1..5]  
5,4,3,2,1
```

```
snoc :: a -> [a] -> [a]  
snoc x xs = xs ++ [x]
```

```
reverse [] = []  
reverse (x:xs) = snoc x (reverse xs)
```


Example: defining `reverse` using `foldr`

Now defining `reverse`: **SOLUTION C**

- Uses `snoc` and `foldr`

```
> reverse [1..5]  
5,4,3,2,1
```

```
snoc :: a -> [a] -> [a]  
snoc x xs = xs ++ [x]
```

```
reverse xs = foldr snoc [] xs
```


General Behavior of `foldr`:

$$\text{foldr } (\#) \ v \ [x_1, x_2, \dots, x_n] = x_1 \# (x_2 \# (\dots (x_n \# v) \dots))$$

map, filter, foldr all in one!

Different approaches to writing a “sum of squares of positives” fn:

A. By combining functions

```
f :: [Int] -> Int
f xs = sum (squares (positives xs))
```

B. By using comprehension

```
f xs = sum [x*x | x <- xs, x > 0]
```

C. By using recursion

```
f [] = 0
f (x:xs)
  | x > 0 = (x*x) + f xs
  | otherwise = f xs
```

D. By using foldr, map, filter

```
f xs = foldr (+) 0 (map sqr (filter pos xs))
  where
    sqr x = x * x
    pos x = x > 0
```


folding left

- Is there a “fold left” function? Yes!

```
foldl :: (a -> b -> a) -> a -> [b] -> a  
foldl f v [] = v  
foldl f v (x:xs) = foldl f (f v x) xs
```

Example Trace

foldl (+) 0 [1,2,3]

General Behavior of `foldl`:

$$\text{foldl } (\#) \ v \ [x_1, x_2, \dots, x_n] = (\dots ((v \# x_1) \# x_2) \dots) \# x_n$$